FQNN PennyLane

Experimental CLI-first implementation of a Fuzzy Quantum Neural Network for tabular classification using Python, PyTorch, and PennyLane.

Repository: github.com/DOKOS-TAYOS/Fuzzy_QNN

The goal is practical experimentation from terminal:

- no need to edit internal Python files to train
- reproducible train/validation/test splits
- GPU-first execution when available
- automatic run artifacts with metrics, checkpoints, and plots

This repository is a research prototype. It does not claim quantum advantage.

Installation

CPU / general setup

Windows:

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

Linux:

```
python -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

GPU-oriented setup

Install the CUDA-enabled PyTorch wheel first, then the GPU extras:

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install --upgrade pip
python -m pip install torch --index-url https://download.pytorch.org/whl/cu121
python -m pip install -r requirements-gpu.txt
```

If `lightning.gpu` still fails to initialize:

```
python -m pip install custatevec-cu12
python -m pip install pennylane-lightning-gpu
```

Main commands

Check runtime on a CPU-only setup:

```
python scripts/check_gpu.py --no-require-gpu
```

Train on CPU:

```
python scripts/train.py --config configs/breast_cancer_cpu.yaml
```

Validate config, dataset, and devices without training:

```
python scripts/train.py --config configs/breast_cancer_cpu.yaml --dry-run
```

Benchmark:

```
python scripts/benchmark_train.py --config configs/breast_cancer_cpu.yaml
```

Evaluate noise robustness:

```
python scripts/evaluate_noise.py --config configs/breast_cancer_cpu.yaml
```

If one day you want to try GPU on Linux with a compatible PennyLane backend, keep using the `*_gpu.yaml` configs.

Re-evaluate a saved run:

```
python scripts/evaluate.py --run-dir runs/breast_cancer_fqnn_gpu/<timestamp>
```

Inspect a sample:

```
python scripts/inspect_sample.py --run-dir runs/breast_cancer_fqnn_gpu/<timestamp> --sample-index 0
```

Useful overrides

Windows multiline example:

```
python scripts/train.py ^
  --config configs/synthetic_gpu.yaml ^
  --max-epochs 5 ^
  --batch-size 8 ^
  --learning-rate 0.005 ^
  --experiment-name synthetic_quick_check ^
  --output-dir runs ^
  --no-require-gpu ^
  --no-prefer-gpu ^
  --torch-device cpu ^
  --quantum-device default.qubit
```

Supported practical overrides include:

- `--epochs`
- `--max-epochs`
- `--batch-size`
- `--learning-rate`
- `--seed`
- `--output-dir`
- `--experiment-name`
- `--dry-run`
- `--require-gpu` / `--no-require-gpu`
- `--prefer-gpu` / `--no-prefer-gpu`
- `--quantum-device`
- `--torch-device`
- `--plot-loss` / `--no-plot-loss`
- `--show-progress` / `--no-show-progress`

Useful behavior:

- `--dry-run` validates the YAML, prepares the dataset split, resolves Torch and PennyLane devices, and prints the planned run summary without training or creating a run folder.
- `--max-epochs` temporarily overrides `training.epochs` from CLI, which is useful for smoke tests and quick checks.
- `--experiment-name` lets you redirect artifacts to a different `runs/&lt;experiment_name&gt;/...` folder without editing YAML.
- If `lightning.gpu` is requested but unavailable and `require_gpu=false`, the CLI explains the fallback device in the startup summary.
- If `require_gpu=true`, the CLI fails with a clearer message explaining how to verify CUDA and PennyLane GPU support or retry with CPU fallback.

Available configs

- `configs/iris_cpu.yaml`
- `configs/breast_cancer_cpu.yaml`
- `configs/wine_cpu.yaml`
- `configs/synthetic_cpu.yaml`
- `configs/iris_gpu.yaml`
- `configs/breast_cancer_gpu.yaml`
- `configs/wine_gpu.yaml`
- `configs/synthetic_gpu.yaml`
- `configs/debug_cpu.yaml`

Supported datasets:

- `iris`
- `breast_cancer`
- `wine`
- `synthetic`

Outputs

Each run creates:

```
runs/<experiment_name>/<timestamp>/
|-- config.yaml
|-- device_info.json
|-- history.csv
|-- history.json
|-- loss_curve.png
|-- accuracy_curve.png
|-- test_metrics.json
|-- metrics.json
|-- model.pt
|-- best_model.pt
|-- rules.json
`-- fuzzy_parameters.json
```

The training loop shows a `tqdm` progress bar and prints final test metrics including:

- test accuracy
- balanced accuracy
- macro F1
- weighted F1
- confusion matrix
- classification report

At startup, `train.py` prints a compact summary with dataset, Torch device, PennyLane device, qubits, rules, classes, and output folder so you can confirm the run before the first epoch starts.

Python package entrypoint

You can also use the package CLI:

```
python -m fuzzy_qnn train --config configs/debug_cpu.yaml
```

Quality checks

```
python -m pytest
ruff check . --fix
```

```
ruff format .
```

Acknowledgment

This work has been supported through QuantumCrip. The research has been funded by ICECyL (Junta de Castilla y Leon) under project `CCTT5/23/BU/0002 (QUANTUMCRIP)`.

QuantumCrip funding logos

ICECyL logo